МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

“БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. Шухова”(БГТУ им. В. Г. Шухова)

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Лабораторная работа №2

по дисциплине: Тестирование программных систем

тема: «Интеграционное и API-тестирование»

Выполнил: ст. группы ВТ-231

Шпак Никита Алексеевич

Проверил:

Доденко Олег Павлович

Белгород, 2026 г.

Лабораторная работа №2 «Интеграционное и API-тестирование»

Цель работы: освоить тестирование REST API на примере открытого публичного сервиса: научиться строить в Postman коллекцию запросов, писать автоматические проверки и анализировать расхождения между документацией и фактическим поведением API.

Тестируемая система: Restful-Booker
Документация API: https://restful-booker.herokuapp.com/apidoc/index.html

1. Тестируемые эндпоинты

Метод	Эндпоинт	Назначение
POST	/auth	Авторизация, получение токена
GET	/booking	Список бронирований
GET	/booking/{id}	Получение бронирования
POST	/booking	Создание бронирования
PUT	/booking/{id}	Полное обновление
PATCH	/booking/{id}	Частичное обновление
DELETE	/booking/{id}	Удаление бронирования
GET	/ping	Проверка работоспособности сервиса

2. Настройка окружения

Создана коллекция LabAPI_Shpak из 30 запросов, сгруппированных по ресурсам /booking, /auth и служебному /ping. **Аутентификация**: токен, получаемый методом POST /auth (логин admin, пароль password123); в защищённых запросах он передаётся в заголовке Cookie: token={{token}}.
Инструмент: Postman — коллекция LabAPI_Shpak, окружение LabAPI_Shpak с переменными baseUrl, token, bookingId; запуск через Collection Runner, одна итерация.
Переменные заполняются автоматически: baseUrl задан при создании окружения (https://restful-booker.herokuapp.com), token извлекается скриптом из ответа POST /auth, bookingId — из ответа POST /booking (в последнем прогоне — 5102).
Для каждого запроса добавлено от трёх до шести проверок на вкладке Scripts: код ответа, время ответа, структура и типы полей; в негативных кейсах дополнительно проверяется отсутствие поля token или bookingid в ответе. Всего — 103 проверки.

3. Автоматизация получения токена.
В POST /auth (Scripts → Post-response) добавлен скрипт:

```
const body = pm.response.json();
if (body && body.token) pm.environment.set("token", body.token);
```

После проверок ответа он берёт token из JSON и сохраняет в {{token}}. Условие if не даёт записать undefined, если в негативном кейсе сервера вернул только {"reason":"Bad credentials"}.

Аналогично в запросе **POST /booking** сохраняется bookingid в переменную {{bookingId}}:

```
const id = pm.response.json().bookingid;
if (id) pm.environment.set("bookingId", id);
```

Сохранение происходит до первого запроса, который использует переменную: {{token}} подставляется в Cookie: token=... для PUT, PATCH, DELETE, а {{bookingId}} — в URL /booking/{{bookingId}}

4. Перечень запросов

№ п/п	№ в Postman	Метод и эндпоинт	Цель	Входные данные	Ожидаемый результат
1	00	GET /ping	Health-check	—	201, быстрый ответ
2	01	POST /auth	Получить токен	admin / password123	200, поле token, токен → env
3	02	POST /auth	Неверный пароль	password	200, reason: Bad credentials, токен не выдан
4	03	POST /auth	Неверный логин	несуществующий пользователь	200, reason: Bad credentials, токен не выдан
5	04	POST /auth	Пустые поля	{}	200, reason: Bad credentials
6	05	POST /auth	Нет поля password	только username	200, reason: Bad credentials
7	10	GET /booking	Список бронирований	—	200, массив {bookingid}
8	11	GET /booking ?checkin= &lastname=	Фильтр по датам и фамилии	несуществующие значения	200, пустой массив
9	12	POST /booking	Создание	Корректный JSON	200, {bookingid, booking}, bookingId → env
10	13	GET /booking /{{bookingId}}	Чтение	bookingId	200, все поля бронирования
11	14	PUT /booking /{{bookingId}}	Полное обновление	Cookie token, полный JSON	200, обновлённый объект

12	15	PATCH /booking /{{bookingId}}	Частичное обновление	Cookie token, {"lastname": ...}	200, изменено одно поле
13	16	PATCH /booking /{{bookingId}}	Без токена	без Cookie	403, изменение отклонено
14	17	PUT /booking /{{bookingId}}	Поддельный токен	token= invalidtoken12345	403, изменение отклонено
15	18	GET /booking /99999999	Несуществующий id	неверный id	404 Not Found
16	19	POST /booking	Некорректный JSON	битый JSON	400
17	20	POST /booking	totalprice строкой	"totalprice": "100"	400
18	21	POST /booking	Без обязательных полей	пустое тело	400
19	22	POST /booking	Пустые строки	firstname: "", lastname: ""	400
20	23	POST /booking	Очень длинные строки	1000+ символов	400
21	24	POST /booking	totalprice = 0	граничное значение	400
22	25	POST /booking	totalprice отрицательный	−100	400
23	26	POST /booking	totalprice очень большой	999999999999	400
24	27	POST /booking	checkout раньше checkin	даты в обратном порядке	400
25	28	POST /booking	Неверный формат даты	15/11/2026	400
26	33	HEAD /booking	Неподдержанный метод	—	404 / 405
27	29	DELETE /booking /{{bookingId}}	Удаление	Cookie token	201, запись удалена
28	31	DELETE /booking /{{bookingId}}	Повторное удаление	Cookie token	404 / 405
29	32	DELETE /booking /99999999	Несуществующий id	—	404
30	30	GET /booking /{{bookingId}}	Повторное чтение	удалённый id	404

Всего 30 запросов, 103 автоматические проверки. Порядок строк — порядок выполнения: Health → Auth → Booking. Номера 06–09 в коллекции Postman не используются (резерв между папками).

5. Таблицы тест-кейсов

1) Позитивные тест-кейсы (цепочка CRUD)

ID	Эндпоинт	Тип	Входные данные	Ожидаемый результат	Фактический результат	Статус
TC-P-01	POST /auth	Позитивный	{"username":"admin","password":"password 123"}	200, тело {"token":"..."}	200, токен получен и записан в {{token}}	Passed
TC-P-02	GET /ping	Позитивный	—	201 (по документации), сервис доступен	201, 515 ms, Content-Type присутствует	Passed
TC-P-03	GET /booking	Позитивный	—	200, непустой массив [{bookingid}]	200, массив из 68 элементов, 326 ms	Passed
TC-P-04	GET /booking/{id}	Позитивный	bookingId = 2382 (из переменной)	200, объект со всеми полями бронирования	200, поля firstname/lastname/totalprice/depositpaid/bookingdates присутствуют, 140ms	Passed
TC-P-05	PUT /booking/{id}	Позитивный	Cookie token={{token}} полный JSON: firstname=Пётр, totalprice=999, additionalneeds=Late checkout	200, объект обновлён полностью	200, firstname=Пётр, totalprice=999, additionalneeds=Late checkout,160 ms	Passed
TC-P-06	PATCH /booking/{id}	Позитивный	Cookie token={{token}}, {"lastname":"Сидоров"}	200, изменено только lastname, прочие поля сохранены	200, lastname=Сидоров, firstname=Пётр, totalprice=999, 140 ms	Passed
TC-P-07	DELETE /booking/{id}	Позитивный	Cookie token, id = {{bookingId}}	201 Created, запись удалена	201, тело Created, 140 мс	Passed

TC-P-08	GET /booking/{id} после удаления	Позитивный (проверка)	bookingId = 2382 (удалён)	404, запись не найдена	404, тело Not Found, 160 ms	Passed
TC-P-09	POST /booking	Позитивный	Корректный JSON (firstname=Иван, totalprice=150, даты 2026-11-10…2026-11-15)	200, создана запись, в ответе bookingid	200, bookingid = {{bookingId}}, объект booking в ответе, 148 мс	Passed
TC-P-10	GET /booking?lastname=ZZZNotFound	Позитивный	query lastname =ZZZNotFound	200, пустой массив []	200, [], 160 ms	Passed

2) Негативные тест-кейсы

ID	Эндпоинт	Тип	Входные данные	Ожидаемый результат	Фактический результат	Статус
TC-N-01	POST /auth	Негативный	{"username":"admin", "password":"wrong_password"}	401 Unauthorized, токен не выдан	200 + {"reason":"Bad credentials"}, поля token нет	Passed
TC-N-02	POST /auth	Негативный	{"username": "not_admin", "password":"password123"}	401 Unauthorized, токен не выдан	200 + reason: Bad credentials, поля token нет	Passed
TC-N-03	POST /auth	Негативный	{} — пустые поля	400 Bad Request либо отказ без выдачи токена	200 + reason: Bad credentials, токен не выдан	Passed
TC-N-04	POST /auth	Негативный	{"username": "admin"} — нет password	400 Bad Request либо отказ без выдачи токена	200 + reason: Bad credentials, токен не выдан	Passed
TC-N-05	PATCH /booking/{{bookingId}}	Негативный	Без заголовка Cookie / Authorization, тело {"firstname":"Хакер"}	401 Unauthorized, изменение отклонено	403 Forbidden, данные не изменены	Passed

TC-N-06	PUT /booking /{{bookingId}}	Негативный	Cookie: token=invalidtoken 12345, полный JSON	401 Unauthorized, изменение отклонено	403 Forbidden, объект booking не возвращён	Passed
TC-N-07	GET /booking /99999999	Негативный	id = 99999999 (не существует)	404 Not Found	404, тело Not Found	Passed
TC-N-08	POST /booking	Негативный	Битый JSON: незакрытый объект (нет } в конце)	400 Bad Request, запись не создана	400, поля bookingid в ответе нет	Passed
TC-N-09	POST /booking	Негативный	"totalprice": "150" — строка вместо числа	400 Bad Request (неверный тип данных)	200, запрос принят, значение сохранено	Failed
TC-N-10	POST /booking	Негативный	Отсутствуют lastname, totalprice, depositpaid, bookingdates	400 Bad Request или корректный отказ без 5xx	500 Internal Server Error, AssertionError: expected [200, 400] to include 500	Failed
TC-N-11	POST /booking	Негативный	"totalprice": -100	400 Bad Request	200, отрицательная стоимость принята	Failed
TC-N-12	POST /booking	Негативный	checkout раньше checkin	400 Bad Request	200, некорректные даты приняты	Failed
TC-N-13	DELETE /booking /99999999	Негативный	Cookie token, id = 99999999	404 Not Found	405 Method Not Allowed	Failed
TC-N-14	DELETE /booking /{{bookingId}}	Негативный	Cookie token, id уже удалён в TC-P-07	404 Not Found	405 Method Not Allowed	Failed
TC-N-15	HEAD /booking	Негативный	Неподдержанный метод	404 или 405 Method Not Allowed	200, AssertionError: expected [404, 405] to include 200	Failed

3) Граничные тест-кейсы

ID	Эндпоинт	Тип	Входные данные	Ожидаемый результат	Фактический результат	Статус

TC-B-01	POST /booking	Граничный	firstname:"", lastname:"", checkin: "", checkout:"", additionalneeds:"" (пустые строки)	400 Bad Request	200, пустые значения приняты	Failed
TC-B-02	POST /booking	Граничный	firstname 1000+символов «A…», lastname 400 символов «B…»	400 или сохранение с ограничением длины	200, строка сохранена целиком, 187 ms	Failed
TC-B-03	POST /booking	Граничный	"totalprice": 0 (нижняя граница)	200, totalprice = 0 сохранён	200, totalprice = 0 в ответе	Passed
TC-B-04	POST /booking	Граничный	"totalprice": -100	400 Bad Request (цена не может быть отрицательной)	200, отрицательная цена сохранена	Failed
TC-B-05	POST /booking	Граничный	"totalprice": 999999999999	400 либо 200 без 5xx, тип значения не потерян	200, значение принято, ответ — валидный JSON	Passed
TC-B-06	POST /booking	Граничный	checkin: "2026-11-15", checkout: "2026-11-10" — checkout раньше checkin	400 Bad Request (нарушено бизнес-правило дат)	200,некорректный период сохранён	Failed
TC-B-07	POST /booking	Граничный	checkin: "15/11/2026", checkout: "not-a-date"	400 Bad Request (неверный формат даты)	200, невалидные даты сохранены, ответ – валидный JSON	Failed
TC-B-08	POST /auth	Граничный	{"username":"","password":""}	400 или 401, токен не выдан	200 + Bad credentials, токена нет	Passed

TC-B-09	POST /auth	Граничный	{"username":"admin"} — нет поля password	400 Bad Request, токен не выдан	200 + отказ, токен не выдан	Passed

4) Покрытие кодов ответов:

Код	Ожидался	Получен	Тест-кейс
200	Да	Да	TC-P-01, P-03…P-06, P-09, P-10, TC-N-01, N-02, N-07, TC-B-01…B-07
201	Да	Да	TC-P-02 (GET /ping), TC-P-07 (DELETE)
400	Да	Да	TC-N-06 (битый JSON)
401	Да	Нет	API не возвращает 401 — вместо него 403 (TC-N-03, N-04)
403	Да	Да	TC-N-03, TC-N-04
404	Да	Да	TC-P-08, TC-N-05
405	Да	Нет	TC-N-13, N-14 — фактически 405 получен, но ожидался 404
500	Да	Да	TC-N-08 (отсутствие обязательных полей)

6. Таблица расхождений фактического и ожидаемого поведения

№	Запрос	Ожидалось (по REST / документации)	Получено	Вывод (дефект / особенность)
1	POST /auth, неверный пароль	401 Unauthorized	200 OK, тело {"reason":"Bad credentials"}	Дефект: ошибка аутентификации маскируется успешным кодом, клиент не может отличить её по коду ответа
2	POST /auth, неверный логин	401 Unauthorized	200 OK + Bad credentials	Дефект: то же, что № 1
3	POST /auth, пустые логин/пароль	400 Bad Request	200 OK + Bad credentials	Дефект: отсутствует валидация обязательных полей
4	POST /auth, отсутствует поле password	400 Bad Request	200 OK + отказ в токене	Дефект: нет проверки обязательных полей тела запроса
5	POST /booking (корректный запрос)	201 Created + заголовок Location	200 OK	Особенность: код не соответствует REST, но зафиксирован в документации API

6	DELETE /booking/{id}	200 OK или 204 No Content, пустое тело	201 Created, тело Created	Особенность: код 201 неверен для удаления
7	PATCH /booking/{id} без токена	401 Unauthorized (+ WWW-Authenticate)	403 Forbidden, тело Forbidden	Дефект/расхождение: некорректно выбран код; 401 в API не используется вовсе
8	PUT /booking/{id} с некорректным токеном	401 Unauthorized	403 Forbidden, тело Forbidden	Дефект/расхождение: поддельный токен неотличим от отсутствующего
9	GET /ping	200 OK	201 Created	Особенность: health-check отвечает нестандартным кодом
10	POST /booking, totalprice строкой "150"	400 Bad Request (неверный тип)	200 OK, значение принято	Дефект: нет проверки типа данных
11	POST /booking без обязательных полей	400 Bad Request	500 Internal Server Error	Дефект: необработанное исключение вместо валидации
12	POST /booking, totalprice = -100	400 Bad Request	200 OK, цена сохранена	Дефект: отрицательная цена принимается
13	POST /booking, пустые строки в полях и датах	400 Bad Request	200 OK, пустые значения сохранены	Дефект: пустые значения допустимы
14	POST /booking, firstname 1000+ символов	400 или ограничение длины	200 OK, строка сохранена целиком	Дефект: нет ограничения длины
15	POST /booking, checkout (10.11) раньше checkin (15.11)	400 Bad Request	200 OK, период сохранён	Дефект: бизнес-правило дат не проверяется
16	POST /booking, дата в формате 15/11/2026 / not-a-date	400 Bad Request	200 OK, некорректные даты приняты	Дефект: нет валидации формата даты
17	POST /booking, totalprice = 999999999999	400 или корректное сохранение	200 OK, значение принято	Особенность: нет проверки верхней границы

18	DELETE /booking/{id} повторно	404 Not Found	405 Method Not Allowed	Дефект: некорректный код для отсутствующего ресурса
19	DELETE /booking/99999999 (несуществующий)	404 Not Found	405 Method Not Allowed	Дефект: то же, выявляет отсутствие корректной обработки 404 на DELETE
20	HEAD /booking	405 Method Not Allowed	200 OK	Дефект/особенность: неподдержанный метод обрабатывается как GET без тела
21	Ответы об ошибках (404, 403)	JSON с описанием ошибки	Plain text Not Found, Forbidden	Расхождение: Content-Type не JSON, машиночитаемого описания нет

Итого: Дефекты – 11, расхождение с документацией – 6, особенность реализации – 4.

Список найденных дефектов (баг-репорты)

BUG-01. 500 Internal Server Error при отсутствии обязательных полей
Тест-кейс: TC-N-08 (запрос 21 Booking: POST /booking без обяз. полей)
Серьёзность: Critical
Шаги:

1. Открыть запрос 21 Booking: POST /booking без обязательных полей.
2. Задать заголовок Content-Type: application/json.
3. В теле оставить только {"firstname": "Толькоимя"} — без lastname, totalprice, depositpaid, bookingdates.
4. Отправить запрос.

Ожидаемый результат: 400 Bad Request с описанием отсутствующих обязательных полей; запись не создаётся.
Фактический результат: 500 Internal Server Error.

BUG-02. Строка в поле totalprice принимается без ошибки
Тест-кейс: TC-N-07 (запрос 20 Booking: POST /booking totalprice строкой)
Серьёзность: Major
Шаги:

1. Открыть запрос 20 Booking: POST /booking totalprice строкой.
2. Задать тело: {"firstname":"Тест","lastname":"Типов","totalprice":"150","depositpaid":true,"bookingdates":{"checkin":"2026-11-10","checkout":"2026-11-15"}}.
3. Отправить запрос.

Ожидаемый результат: 400 Bad Request — неверный тип данных (totalprice должен быть числом).
Фактический результат: 200 OK, бронирование создано, строковое значение сохранено; в Console выведено «Дефект: totalprice строкой принят без ошибки валидации».

BUG-03. Отрицательное значение totalprice сохраняется
Тест-кейс: TC-B-04 (запрос 25 Booking: POST /booking totalprice отриц.)
Серьёзность: Major
Шаги:

1. Открыть запрос 25 Booking: POST /booking totalprice отрицательный.
2. Задать тело: {"firstname":"Минус","lastname":"Минусов","totalprice":-100,"depositpaid":true,"bookingdates":{"checkin":"2026-11-10","checkout":"2026-11-15"}}.
3. Отправить запрос.

Ожидаемый результат: 400 Bad Request — стоимость бронирования не может быть отрицательной.
Фактический результат: 200 OK, бронирование с totalprice = -100 создано.

BUG-04. Не проверяется соотношение дат checkin и checkout
Тест-кейс: TC-B-06 (запрос 27 Booking: POST /booking checkout раньше checkin)
Серьёзность: Major
Шаги:

1. Открыть запрос 27 Booking: POST /booking checkout раньше checkin.
2. Задать тело: "bookingdates":{"checkin":"2026-11-15","checkout":"2026-11-10"} (выезд раньше заезда).
3. Отправить запрос.

Ожидаемый результат: 400 Bad Request — дата checkout не может быть раньше checkin.
Фактический результат: 200 OK, бронирование с некорректным периодом создано; даты не проверяются.

BUG-05. Невалидный формат даты не отклоняется
Тест-кейс: TC-B-07 (запрос 28 Booking: POST /booking неверный формат даты)
Серьёзность: Major
Шаги:

1. Открыть запрос 28 Booking: POST /booking неверный формат даты.
2. Задать тело: "bookingdates":{"checkin":"15/11/2026","checkout":"not-a-date"}.
3. Отправить запрос.

Ожидаемый результат: 400 Bad Request — даты обязаны соответствовать формату YYYY-MM-DD.
Фактический результат: 200 OK, некорректные даты приняты (в текущем прогоне).

BUG-06. Неверные учётные данные возвращают 200 OK вместо 401
Тест-кейс: TC-N-01, TC-N-02 (запросы 02 Auth: неверный пароль, 03 Auth: неверный логин)
Серьёзность: Major
Шаги:

1. Открыть запрос 02 Auth: POST /auth неверный пароль.
2. Задать тело {"username":"admin","password":"wrong_password"}.
3. Отправить запрос.
4. Повторить п. 1–3 для запроса 03 Auth с телом {"username":"not_admin","password":"password123"}.

Ожидаемый результат: 401 Unauthorized, токен не выдан.
Фактический результат: 200 OK, тело {"reason":"Bad credentials"}, токен не выдан. Клиент не может определить отказ по коду ответа.

BUG-07. Вместо 401 возвращается 403 при отсутствии и при некорректном токене
Тест-кейс: TC-N-03, TC-N-04 (запросы 16 PATCH без токена, 17 PUT некорректный токен)
Серьёзность: Major
Шаги:

1. Открыть запрос 16 Booking: PATCH /booking/{{bookingId}} без токена — без заголовков Cookie и Authorization.
2. Задать тело {"firstname":"Хакер"}, отправить запрос.
3. Открыть запрос 17 Booking: PUT /booking/{{bookingId}} некорректный токен, задать Cookie token=invalidtoken12345 и полный JSON.
4. Отправить запрос.

Ожидаемый результат: 401 Unauthorized с заголовком WWW-Authenticate; изменение отклонено.
Фактический результат: 403 Forbidden, тело Forbidden, заголовок WWW-Authenticate отсутствует; код 401 не возвращается ни в одном сценарии API.

BUG-08. DELETE отсутствующей записи возвращает 405 вместо 404
Тест-кейс: TC-N-09, TC-N-10 (запросы 31 повторный DELETE, 32 DELETE /99999999)
Серьёзность: Minor
Шаги:

1. Создать бронирование (bookingId = 2382) и удалить его — получен 201 Created.
2. Повторно отправить DELETE {{baseUrl}}/booking/2382 с Cookie token={{token}}.
3. Отправить DELETE {{baseUrl}}/booking/99999999 (id, которого не существует).

Ожидаемый результат: 404 Not Found — запись отсутствует.
Фактический результат: 405 Method Not Allowed в обоих случаях; клиент получает ложное сообщение о неподдержанном методе.

BUG-09. Пустые строки в обязательных полях принимаются
Тест-кейс: TC-B-01 (запрос 22 Booking: POST /booking пустые строки)
Серьёзность: Minor
Шаги:

1. Открыть запрос 22 Booking: POST /booking пустые строки.
2. Задать тело: {"firstname":"","lastname":"","totalprice":100,"depositpaid":true,"bookingdates":{"checkin":"","checkout":""},"additionalneeds":""}.
3. Отправить запрос.

Ожидаемый результат: 400 Bad Request — обязательные строковые поля и даты не могут быть пустыми.
Фактический результат: 200 OK, бронирование создано с пустыми значен.

BUG-10. Нет ограничения длины строковых полей
Тест-кейс: TC-B-02 (запрос 23 Booking: POST /booking длинные строки 1000+)
Серьёзность: Minor
Шаги:

1. Открыть запрос 23 Booking: POST /booking очень длинные строки 1000+.
2. Задать firstname — 1000+ символов «A», lastname — 400 символов «B».
3. Отправить запрос.

Ожидаемый результат: 400 Bad Request либо обрезка значения до допустимой длины.
Фактический результат: 200 OK, строка сохранена (время ответа 187 ms).

Особенности реализации:
OBS-01. Ошибки возвращаются в plain text вместо JSON
Тест-кейс: TC-N-07, TC-N-05 (запросы 18 GET /booking/99999999, 16 PATCH без токена)
Серьёзность: не дефект — особенность API, зафиксирована в таблице расхождений под № 21.

Шаги:

1. Отправить GET {{baseUrl}}/booking/99999999.
2. Отправить PATCH {{baseUrl}}/booking/2382 без токена.
3. Проверить тело и заголовок Content-Type ответов.

Ожидаемый результат: JSON-объект с кодом и описанием ошибки, Content-Type: application/json.

Фактический результат: plain text Not Found и Forbidden, Content-Type не JSON.

7. Результаты запуска Collection Runner

Выполнен один полный прогон коллекции (30 запросов, одна итерация) с окружением LabAPI_Shpak.

Показатель	**Фактическое значение**
Количество запросов	30
Автоматические проверки	103
Успешные проверки	100
Неуспешные проверки	3
Пропущено	0
Ошибки выполнения Runner	0
Длительность	8,813 с
Среднее время ответа	206 мс

Результат. Неуспешные утверждения находятся в двух запросах: № 21 (POST /booking без обязательных полей) и № 33 (HEAD /booking). Три непрошедших проверки не равны трём разным дефектам: в запросе № 21 две проверки фиксируют один и тот же HTTP-код 500 — «Ответ получен (400 или 200)» и «Нет 500 (сервер не падает)», то есть один дефект валидации отмечен дважды. Третье падение — в запросе № 33: проверка «Статус 404 или 405» не пройдена, потому что неподдержанный метод HEAD обработан как GET и вернул 200.

8. Скриншоты прогона коллекции в Collection Runner.

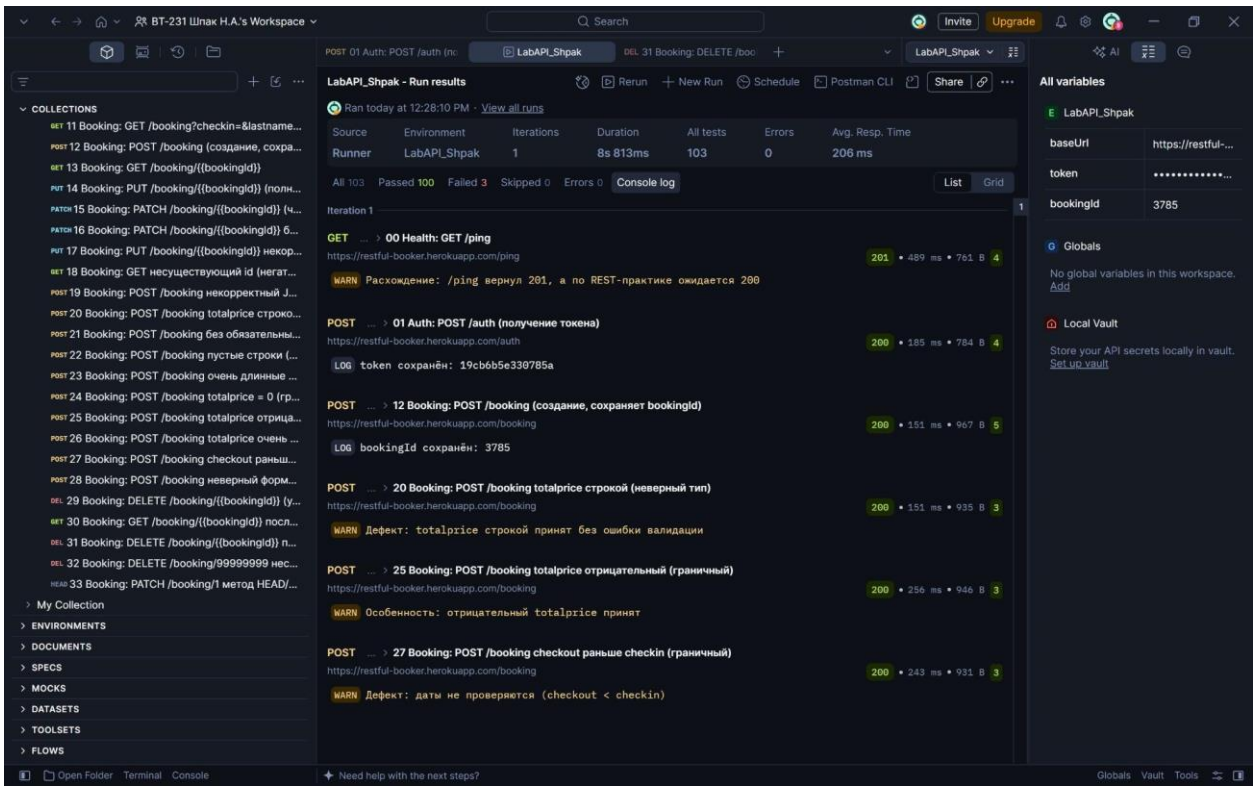


Рис.1. Итог прогона и ход выполнения: 103 проверки, 100 Passed, 3 Failed, 0 Skipped

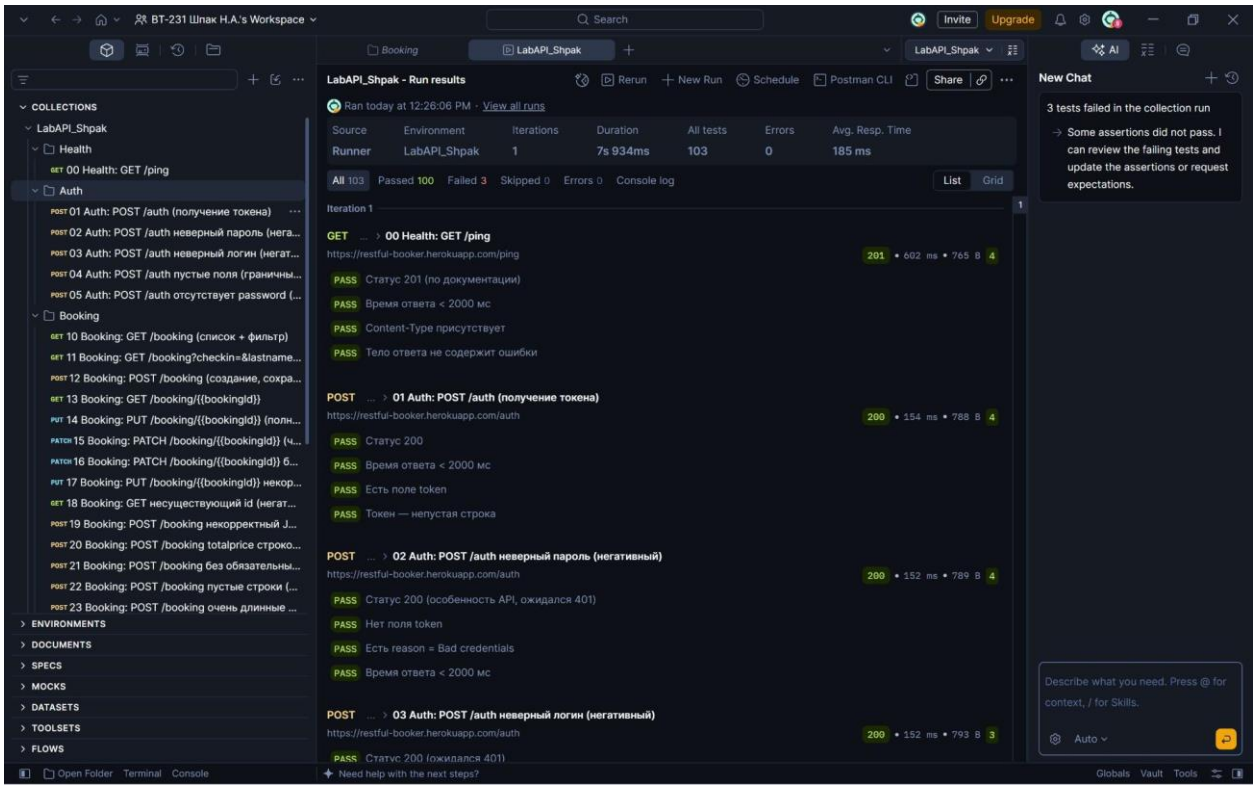


Рис.2. Проверки запросов Health и Auth: GET /ping и POST /auth

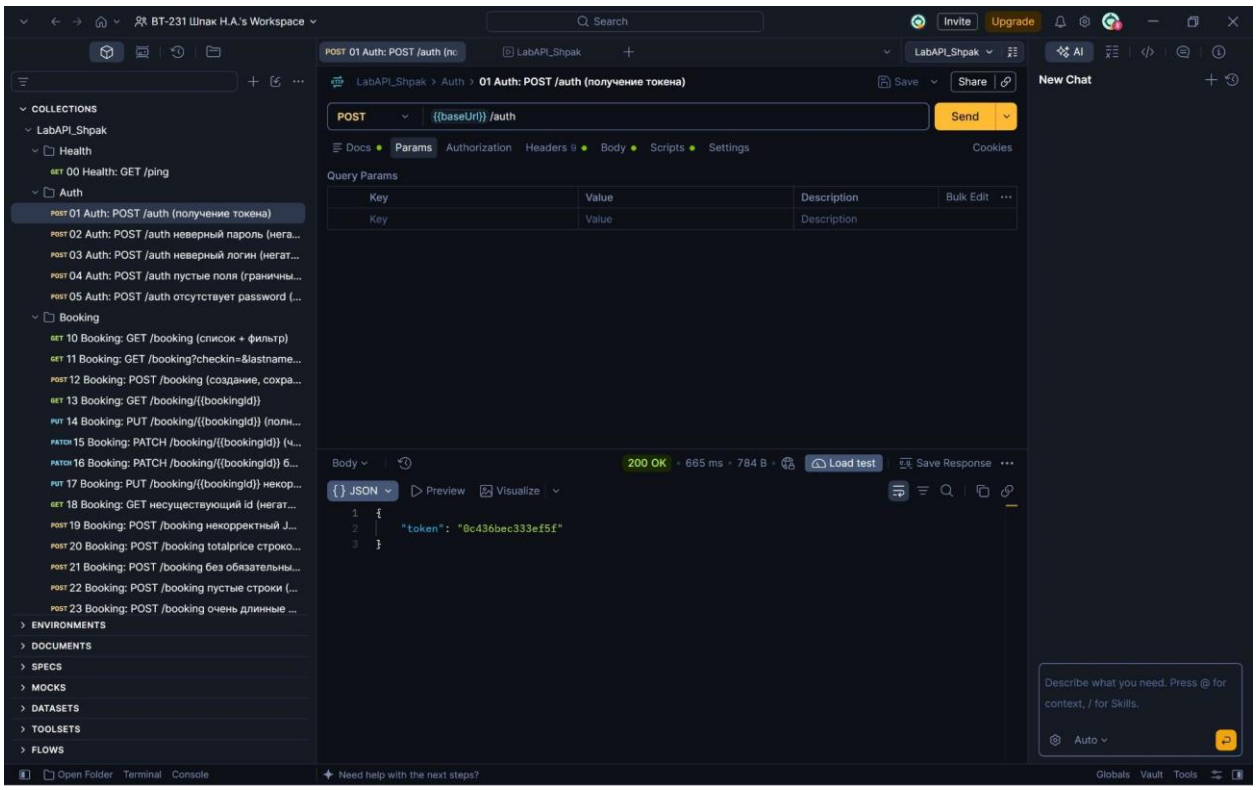


Рис.3. Запрос 01 Auth: POST /auth — получение токена, 200 OK

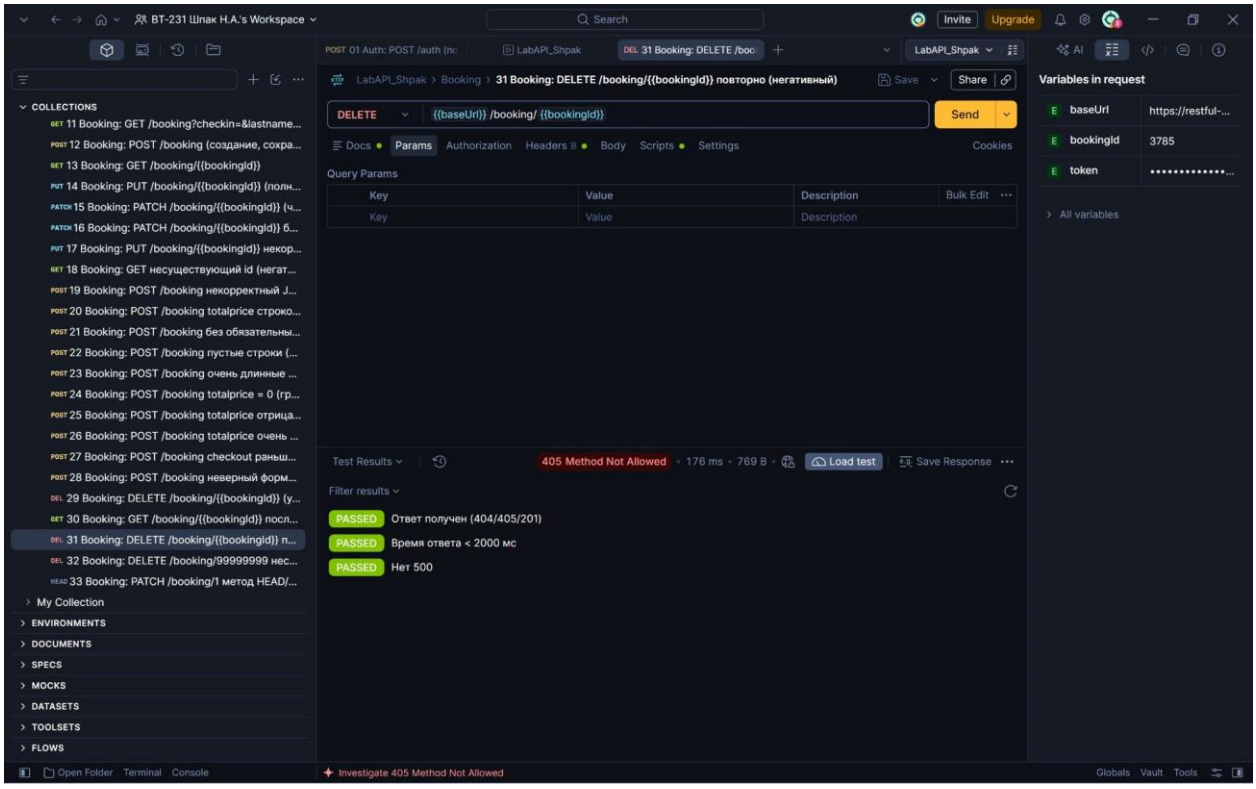


Рис.4. Запрос 31 Booking: повторное DELETE /booking/{{bookingId}} — 405, 3/3 PASSED

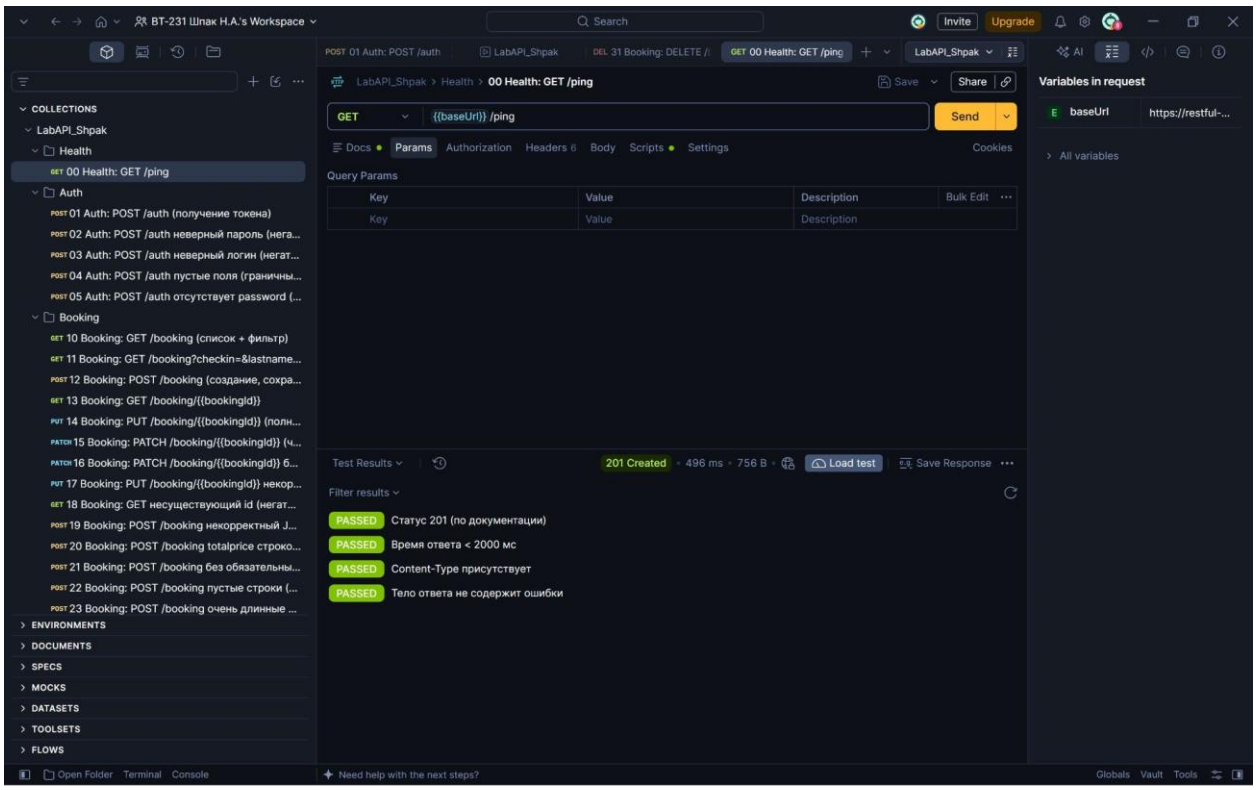


Рис.5. Запрос 00 Health: GET /ping — 201 Created, 4/4 PASSED

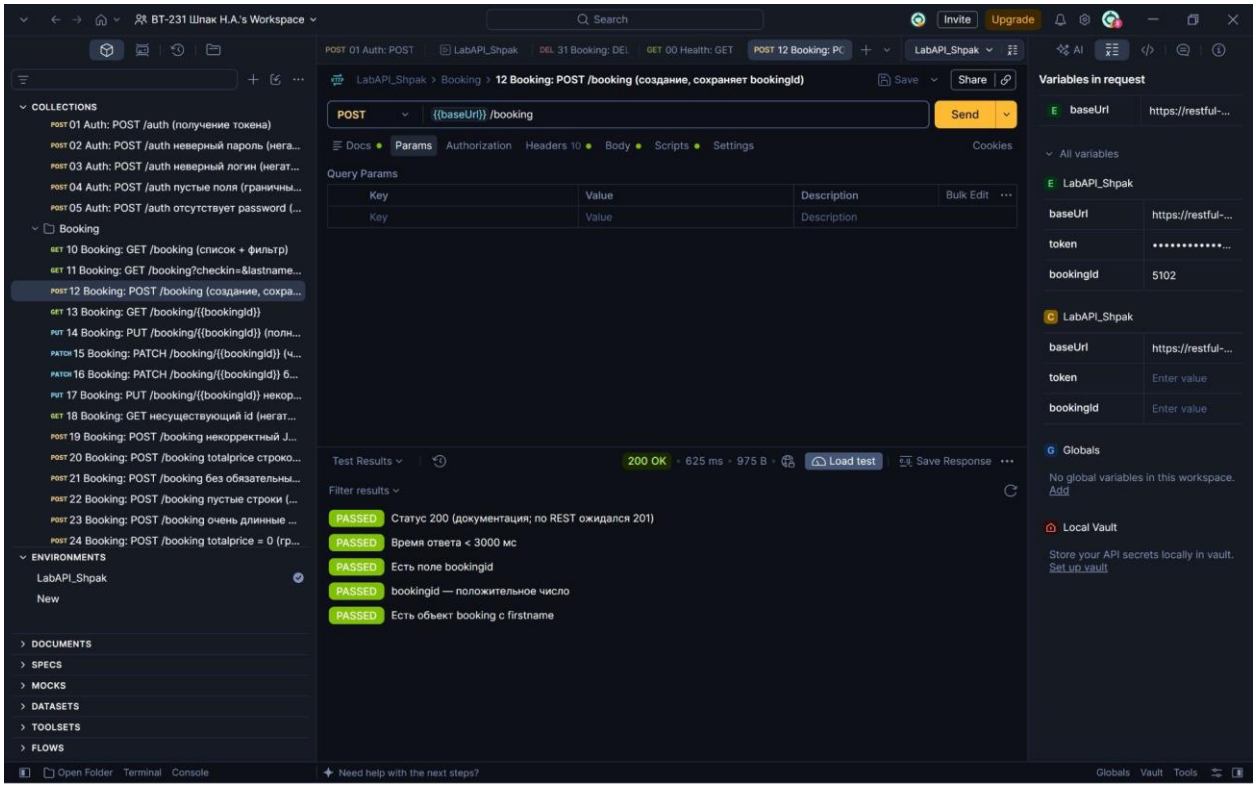


Рис.6. Запрос 12 Booking: POST /booking — 200 OK, 5/5 PASSED

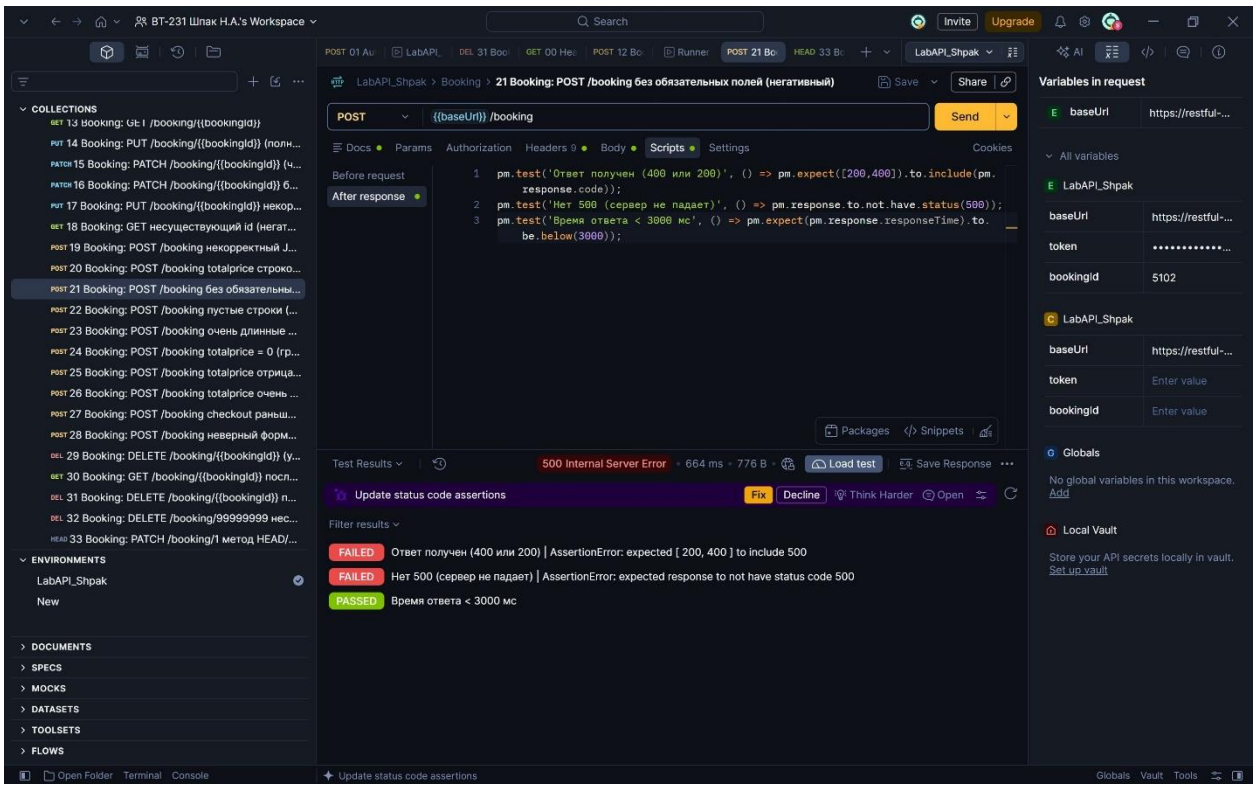


Рис.7. Запрос 21 Booking: POST /booking без обязательных полей — скрипт проверок на вкладке Scripts

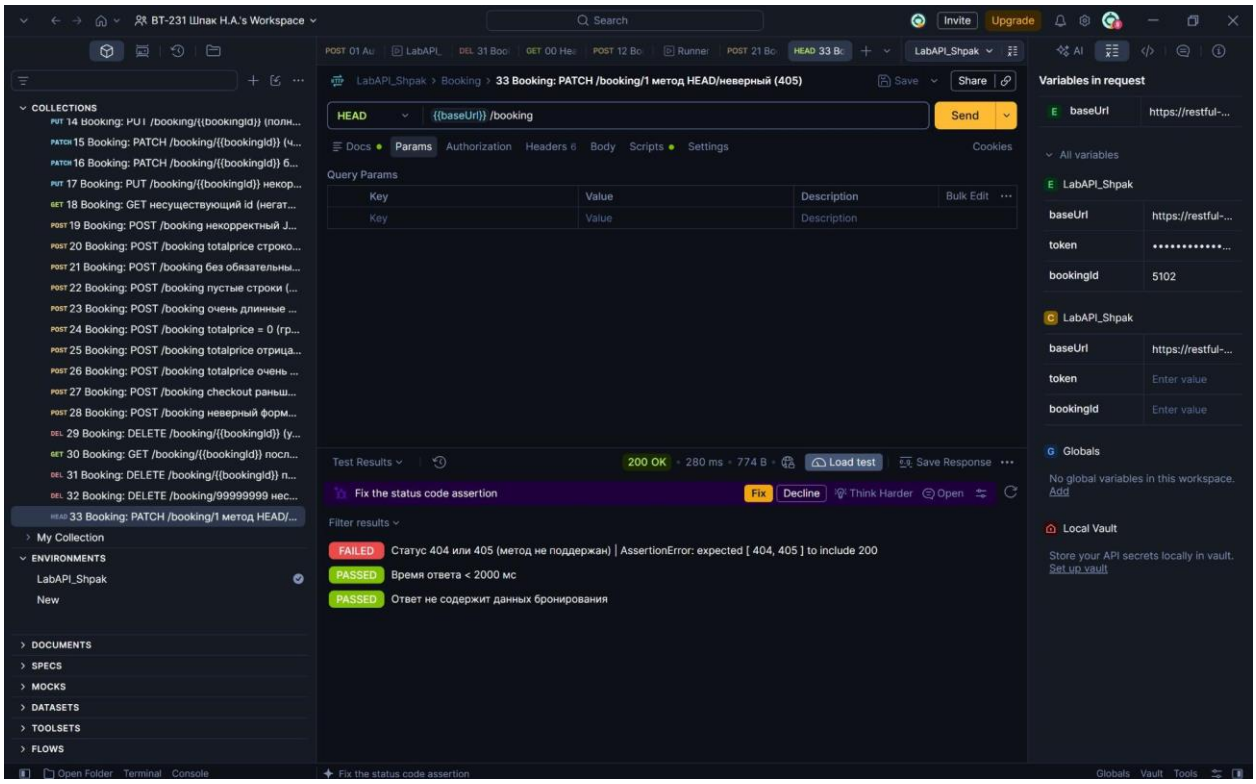


Рис.8. Сбой № 33: HEAD /booking — 200 OK вместо 405, 2/3

9. Экспортированные файлы коллекции и окружения

В папке с лабораторной работой приложены файлы:

1. LabAPI_Shpak.postman_collection.json – 30 запросов, папки Health/Auth/Booking, тела запросов, все скрипты на вкладке Scripts, описание коллекции
2. LabAPI_Shpak.postman_environment.json – переменные baseUrl, token, bookingId

10. Выводы

В ходе лабораторной работы для Restful-Booker построена коллекция LabAPI_Shpak из 30 запросов, покрывающая ресурсы /ping, /auth и /booking и полный CRUD-цикл бронирования. Контрольные точки оформлены как тесты на вкладке Scripts, что позволило получить 103 автоматические проверки.

Один полный прогон Collection Runner дал результат: 100 проверок Passed, 3 Failed, 0 Skipped, 0 ошибок выполнения, длительность 8,813 с, среднее время ответа 206 мс. Падения сосредоточены в двух запросах: POST /booking без обязательных полей дважды фиксирует код 500, а HEAD /booking возвращает 200 вместо ожидаемых 404/405.

Основные выводы по качеству API:

1. Валидация тела запроса практически отсутствует. Сервис принимает строку в totalprice, отрицательную стоимость, пустые строки, даты в формате 15/11/2026, период с checkout раньше checkin и значения длиной 1000+ символов. При отсутствии обязательных полей вместо 400 возвращается 500 — необработанное исключение, наиболее критичный дефект (BUG-01).
2. Нестандартные, но стабильные коды — 200 на создание, 201 на удаление и на health-check — документированы и не мешают интеграции, поэтому отнесены к особенностям реализации, а не к дефектам.

Итог: 21 расхождение с ожидаемым поведением, из них 10 дефектов (1 Critical, 6 Major, 3 Minor) и 1 особенность.

Таким образом, работа показала, что API-тестирование выявляет не только функциональные ошибки, но и несоответствия контракту и стандартам: именно к этой категории относится большинство найденных расхождений.